

Exam23 参考答案

姓名: Copilot 学号: 123456

2025 年 12 月 4 日

第 1 题

题目: 摊销复杂度和渐进复杂度的定义, 摊销分析和渐进分析的意义? NP-hard 是否是指 NP 里最难的问题? NP 和 NP-hard 的定义?

(1) 摊销复杂度和渐进复杂度的定义

渐进复杂度 (Asymptotic Complexity):

- **定义:** 描述算法在输入规模 n 趋向无穷大时的性能表现
- 使用渐进记号: O (上界)、 Ω (下界)、 Θ (紧确界)
- 关注最坏情况、平均情况或最好情况下单次操作的时间/空间消耗
- 例如: 快速排序的最坏情况时间复杂度为 $O(n^2)$, 平均情况为 $O(n \log n)$

摊销复杂度 (Amortized Complexity):

- **定义:** 在最坏情况下, 对一个数据结构进行 n 次连续操作所需的平均时间
- 计算方法: 总代价 / 操作次数
- 不是概率平均, 而是对操作序列的整体分析
- 某些单次操作可能代价很高, 但通过分摊到整个序列, 平均代价较低
- 例如: 动态数组扩容, 单次扩容 $O(n)$, 但 n 次插入的摊销复杂度为 $O(1)$

(2) 摊销分析和渐进分析的意义

渐进分析的意义：

- 提供算法效率的理论保证
- 忽略常数因子和低阶项，关注增长趋势
- 便于比较不同算法的优劣
- 适用于单次操作的性能评估

摊销分析的意义：

- 更准确地反映数据结构在实际使用中的性能
- 避免过度悲观的估计（仅考虑最坏情况）
- 适用于分析操作序列的整体效率
- 三种主要方法：聚集法、记账法、势能法
- 实际应用：动态表、斐波那契堆、Splay 树等

(3) NP-hard 是否是指 NP 里最难的问题？

答案：不完全准确。

更准确的理解：

- NP-hard 问题**不一定**属于 NP 类
- NP-hard 是指“至少和 NP 中最难的问题一样难”的问题
- NP-hard 问题可能比 NP 问题更难（例如，可能不在 NP 中）
- NP-Complete 问题才是“NP 中最难的问题”（既在 NP 中，又是 NP-hard）

(4) NP 和 NP-hard 的定义

NP 类 (Nondeterministic Polynomial time)：

- **定义：**能够在多项式时间内**验证**一个给定解（证书）是否正确的判定问题类
- **形式化：**如果存在多项式时间算法 V ，对于任意实例 x ， x 有解当且仅当存在证书 y （长度为 $|x|$ 的多项式）使得 $V(x, y) = \text{Yes}$
- **例子：**哈密尔顿回路问题、子集和问题、SAT 问题

- 注意: $P \subseteq NP$ (能快速求解的一定能快速验证)

NP-Hard 类:

- **定义:** 至少和 NP 中最难的问题一样难的问题类
- **形式化:** 问题 H 是 NP-Hard, 如果 NP 中的每个问题都可以在多项式时间内归约到 H
- **关键点:** NP-Hard 问题**不要求**属于 NP
- 例子: 停机问题 (不在 NP 中)、优化版 TSP (求最短路径, 而非判定是否存在)

NP-Complete 类 (NPC):

- 同时满足: (1) 属于 NP, (2) 是 NP-Hard
- 即: $NPC = NP \cap NP\text{-Hard}$
- 是 NP 类中最难的判定问题

关系图示:

- $P \subseteq NP$
- $NPC = NP \cap NP\text{-Hard}$
- 存在 NP-Hard 问题不在 NP 中
- $P = NP$ 问题至今未解决

第 2 题

题目: 求递归方程 $T(n) = 4T(n/2) + n^2 \lg n$ 的上界

解答:

使用主定理求解。递归方程形式:

$$T(n) = 4T(n/2) + n^2 \lg n$$

主定理参数:

- $a = 4, b = 2$
- $f(n) = n^2 \lg n$
- $n^{\log_b a} = n^{\log_2 4} = n^2$

比较 $f(n)$ 与 $n^{\log_b a}$:

$$f(n) = n^2 \lg n \quad \text{vs} \quad n^{\log_b a} = n^2$$

显然 $f(n) = n^2 \lg n = \Theta(n^2 \lg n)$

这对应主定理的扩展情况:

$$f(n) = \Theta(n^{\log_b a} \log^k n), \quad k = 1$$

根据主定理扩展: 当 $f(n) = \Theta(n^{\log_b a} \log^k n)$ 时,

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

因此:

$$T(n) = \Theta(n^2 \log^2 n)$$

结论: 递归方程的上界为 $O(n^2 \log^2 n)$ 。

验证: 也可以用迭代法验证:

$$\begin{aligned} T(n) &= 4T(n/2) + n^2 \lg n \\ &= 4[4T(n/4) + (n/2)^2 \lg(n/2)] + n^2 \lg n \\ &= 16T(n/4) + n^2 \lg(n/2) + n^2 \lg n \\ &= 16T(n/4) + n^2 (\lg n - 1) + n^2 \lg n \\ &= 16T(n/4) + 2n^2 \lg n - n^2 \end{aligned}$$

继续展开 k 层:

$$T(n) = 4^k T(n/2^k) + \sum_{i=0}^{k-1} 4^i \cdot \frac{n^2}{2^{2i}} \lg(n/2^i)$$

当 $k = \lg n$ 时, $n/2^k = 1$, 总和约为 $O(n^2 \log^2 n)$ 。

第 3 题

题目: 一个数组储存了 n 个不同元素, 其中各元素可能是负整数、零、正整数, 且按照元素大小递增排序, 现需要找到一个元素满足 $A[i] = i$, 请找到一个时间复杂度尽可能小的算法实现该需求。

算法设计思路

这是一个在有序数组中查找特殊元素的问题, 可以使用**二分查找**。

关键观察:

- 数组按递增顺序排序
- 要找满足 $A[i] = i$ 的索引 i
- 定义函数 $f(i) = A[i] - i$ ，我们要找 $f(i) = 0$ 的位置

单调性分析：

虽然 $f(i) = A[i] - i$ 不一定单调，但我们可以利用以下性质：

- 如果 $A[mid] > mid$ ：由于数组元素各不相同且递增，对于 $j > mid$ ，有：

$$A[j] \geq A[mid] + (j - mid) > mid + (j - mid) = j$$

因此右半部分不可能有解

- 如果 $A[mid] < mid$ ：类似地，左半部分不可能有解
- 如果 $A[mid] = mid$ ：找到答案

因此可以使用二分查找。

算法伪代码

Algorithm 查找固定点 $A[i] = i$

Input: 递增有序数组 $A[1..n]$ （元素各不相同）

Output: 满足 $A[i] = i$ 的索引 i ，若不存在返回 -1

```

1:  $left \leftarrow 1, right \leftarrow n$ 
2: while  $left \leq right$  do
3:    $mid \leftarrow \lfloor (left + right) / 2 \rfloor$ 
4:   if  $A[mid] = mid$  then
5:     return  $mid$                                      ▷ 找到固定点
6:   else if  $A[mid] > mid$  then
7:      $right \leftarrow mid - 1$                              ▷ 在左半部分查找
8:   else                                           ▷  $A[mid] < mid$ 
9:      $left \leftarrow mid + 1$                              ▷ 在右半部分查找
10: return -1                                           ▷ 不存在固定点

```

时间复杂度分析

- 二分查找每次将搜索范围减半
- 最多需要 $\lceil \log_2 n \rceil$ 次比较

- 每次比较和索引计算都是 $O(1)$

时间复杂度： $O(\log n)$

空间复杂度： $O(1)$ （仅使用常数额外空间）

正确性说明

示例：

- 数组 $A = [-10, -5, 0, 3, 7]$ （索引从 1 开始）
- $A[1] = -10 < 1$, $A[2] = -5 < 2$, $A[3] = 0 < 3$, $A[4] = 3 < 4$, $A[5] = 7 > 5$
- 不存在固定点
- 数组 $A = [-10, -5, 2, 3, 7]$ （索引从 1 开始）
- $A[3] = 2 < 3$, $A[4] = 3 < 4$
- 如果从 0 开始索引： $A = [-10, -5, 2, 3, 7]$ ，则 $A[2] = 2$, $A[3] = 3$ 都是固定点

算法能够高效找到任一固定点，时间复杂度远小于线性扫描的 $O(n)$ 。

第 4 题

题目：驾驶一辆汽车出游，汽车油箱的最大油量为 c ，且汽车的耗油量恒定为 c/k （每公里耗油量），沿路上有许多加油站，最远的加油站距离起始点的距离 $d > k$ ，请设计一个贪心算法，要求汽车的加油次数最少。

问题分析

理解题意：

- 油箱容量： c
- 每公里耗油： c/k
- 满油可行驶距离： $c \div (c/k) = k$ 公里
- 目标：从起点到目的地（距离 $d > k$ ），加油次数最少

贪心策略：每次加油后，尽可能行驶最远距离再加油（即在能到达的最远加油站加油）。

贪心选择性质证明

贪心选择：在当前位置，选择能到达的最远的加油站。

证明：

假设最优解在位置 p 加油后，选择在位置 s_1 加油（ s_1 不是从 p 能到达的最远加油站）。设 s_2 是从 p 能到达的最远加油站，且 $s_1 < s_2 \leq p + k$ 。

考虑替代方案：在 p 加油后直接到 s_2 加油。

- 原方案中从 s_1 能到达的所有位置，从 s_2 也能到达（因为 $s_2 > s_1$ ）
- 替代方案的加油次数不会增加
- 后续的行程从 s_2 开始比从 s_1 开始更有优势

因此，贪心选择是安全的。

优化子结构

如果我们已经贪心地选择了前 m 个加油站，那么从第 m 个加油站到目的地的问题，仍然是一个最少加油次数问题，具有相同的结构。

算法伪代码

Algorithm 最少加油次数贪心算法

Input: 加油站位置数组 $stations[1..n]$ （按距离起点的位置递增排序），目的地距离 d ，满油行驶距离 k

Output: 最少加油次数，若无法到达返回 -1

```

1:  $currentPos \leftarrow 0$                                 ▷ 当前位置（起点）
2:  $fuelCount \leftarrow 0$                                 ▷ 加油次数
3:  $i \leftarrow 1$                                          ▷ 下一个考虑的加油站索引
4: while  $currentPos + k < d$  do                            ▷ 还未能直接到达目的地
5:    $nextPos \leftarrow currentPos$                         ▷ 下一个加油位置
6:   while  $i \leq n$  and  $stations[i] \leq currentPos + k$  do
7:      $nextPos \leftarrow stations[i]$                     ▷ 更新为能到达的最远加油站
8:      $i \leftarrow i + 1$ 
9:   if  $nextPos = currentPos$  then                        ▷ 没有可达的加油站
10:    return -1                                           ▷ 无法到达目的地
11:    $currentPos \leftarrow nextPos$                         ▷ 移动到该加油站并加油
12:    $fuelCount \leftarrow fuelCount + 1$ 
13: return  $fuelCount$ 

```

时间复杂度分析

- 外层 while 循环：最多执行 $O(n)$ 次（每次至少考虑一个加油站）
- 内层 while 循环：每个加油站最多被访问一次
- 总时间复杂度： $O(n)$

空间复杂度： $O(1)$ （仅使用常数额外空间）

示例

假设 $k = 400$ 公里，加油站位置：[100, 200, 300, 400, 500, 600]，目的地 $d = 550$ 。

1. 起点 $pos = 0$ ，能到达 [100, 200, 300, 400]，选择 400，加油次数 = 1
2. $pos = 400$ ，能到达 [500, 600]，但目的地 550 在 $pos + k = 800$ 范围内
3. 无需再加油，直接到达目的地

最少加油次数：1 次。

第 5 题

题目：有 n 个礼品的总价值为 w ，现在将这些礼品分配给两个孩子，要求找到一个分配方案，使得最后两个人得到的礼品价值相等，请设计一个动态规划算法验证是否存在这种方案。

问题分析

这是一个子集和问题（Subset Sum Problem）的变体：

- 要使两个孩子得到的价值相等，每人应得 $w/2$
- 等价于：是否存在一个子集，其价值和为 $w/2$
- 前提条件： w 必须是偶数（否则不可能均分）

算法设计思路

使用动态规划求解子集和问题：

状态定义：

$$dp[i][j] = \begin{cases} \text{True} & \text{前 } i \text{ 个礼品中存在子集，和为 } j \\ \text{False} & \text{否则} \end{cases}$$

目标：计算 $dp[n][w/2]$

优化子结构

对于第 i 个礼品（价值 v_i ），有两种选择：

1. **不选**: $dp[i][j] = dp[i-1][j]$
2. **选**: $dp[i][j] = dp[i-1][j-v_i]$ （需要 $j \geq v_i$ ）

只要有一种选择可行， $dp[i][j]$ 就为真。

递推方程

边界条件：

- $dp[0][0] = \text{True}$ （0 个礼品，和为 0）
- $dp[0][j] = \text{False}$, $\forall j > 0$ （0 个礼品，和不能为正）

递推关系：对于 $i \geq 1, j \geq 0$:

$$dp[i][j] = dp[i-1][j] \text{ OR } (j \geq v_i \text{ AND } dp[i-1][j-v_i])$$

算法伪代码

Algorithm 礼品均分验证（动态规划）

Input: 礼品价值数组 $v[1..n]$ ，总价值 w

Output: 是否存在均分方案（True/False）

```

1: if  $w \bmod 2 = 1$  then                                     ▷ 总价值为奇数
2:   return False                                             ▷ 无法均分
3:  $target \leftarrow w/2$ 
4: 初始化  $dp[0..n][0..target]$  全部为 False
5:  $dp[0][0] \leftarrow \text{True}$ 
6: for  $i = 1$  to  $n$  do
7:   for  $j = 0$  to  $target$  do
8:      $dp[i][j] \leftarrow dp[i-1][j]$                            ▷ 不选第  $i$  个礼品
9:     if  $j \geq v[i]$  and  $dp[i-1][j-v[i]]$  then
10:       $dp[i][j] \leftarrow \text{True}$                                ▷ 选第  $i$  个礼品
11: return  $dp[n][target]$ 

```

时间复杂度分析

- 状态数量: $O(n \cdot w)$

- 每个状态的计算: $O(1)$
- 总时间复杂度: $O(nw)$

空间复杂度: $O(nw)$

空间优化: 可以使用滚动数组优化到 $O(w)$:

Algorithm 空间优化版本

```

1: 初始化  $dp[0..target]$  全部为 False
2:  $dp[0] \leftarrow \text{True}$ 
3: for  $i = 1$  to  $n$  do
4:   for  $j = target$  downto  $v[i]$  do                                ▷ 逆序遍历
5:     if  $dp[j - v[i]]$  then
6:        $dp[j] \leftarrow \text{True}$ 
7: return  $dp[target]$ 

```

优化后空间复杂度: $O(w)$

第 6 题

题目: 回忆摊销分析中讲的表扩张问题, 解释为什么不能在装载因子小于 $1/2$ 时就收缩表, 请设计一个可行的新方案。

为什么不能在装载因子小于 $1/2$ 时就收缩表

装载因子定义: $\alpha = \frac{\text{元素个数}}{\text{表大小}}$

问题分析:

如果在 $\alpha < 1/2$ 时立即收缩表 (缩小一半), 会导致**抖动 (thrashing)** 现象:

示例:

1. 假设表大小为 m , 元素个数为 $m/2$, 装载因子 $\alpha = 0.5$
2. 插入一个元素: $n = m/2 + 1 > m/2$, $\alpha > 0.5$
3. 删除一个元素: $n = m/2 < m/2$, 触发收缩, 表大小变为 $m/2$, $\alpha = 1$
4. 再插入一个元素: $n = m/2 + 1 > m/2$, 触发扩展, 表大小变为 m
5. 再删除一个元素: $n = m/2$, 又触发收缩...

在元素数量在 $m/2$ 附近波动时, 每次插入/删除都会触发扩展/收缩, 导致:

- 每次操作都是 $O(m)$ ，而不是摊销 $O(1)$
- 摊销分析失效
- 性能严重下降

可行的新方案

改进策略：使用不同的扩展和收缩阈值，增加**迟滞** (hysteresis)。

方案设计：

- **扩展：**当 $\alpha \geq 1$ 时，表大小翻倍 ($m \rightarrow 2m$)
- **收缩：**当 $\alpha \leq 1/4$ 时，表大小减半 ($m \rightarrow m/2$)

方案优点：

1. 避免抖动：扩展和收缩阈值之间有足够的间隔
2. 在 $\alpha = 1/4$ 收缩后， α 变为 $1/2$
3. 在 $\alpha = 1$ 扩展后， α 变为 $1/2$
4. 需要连续多次操作才能触发下一次扩展/收缩

摊销分析

使用**势能法**分析：

势能函数定义：

$$\Phi(T) = \begin{cases} 2n - m & \text{if } \alpha \geq 1/2 \\ m/2 - n & \text{if } \alpha < 1/2 \end{cases}$$

其中 n 是元素个数， m 是表大小。

插入操作摊销代价分析：

情况 1：插入后 $\alpha < 1$ (不扩展)

- 实际代价： $c_i = 1$
- 势能变化： $\Delta\Phi = 2$ (假设 $\alpha \geq 1/2$)
- 摊销代价： $\hat{c}_i = c_i + \Delta\Phi = 1 + 2 = 3$

情况 2：插入后 $\alpha = 1$ (触发扩展)

- 实际代价： $c_i = n + 1$ (复制 n 个元素 + 插入 1 个)

- 扩展前: $n = m$, $\Phi = 2m - m = m$
- 扩展后: $n = m + 1$, $m' = 2m$, $\Phi' = 2(m + 1) - 2m = 2$
- 势能变化: $\Delta\Phi = 2 - m$
- 摊销代价: $\hat{c}_i = (m + 1) + (2 - m) = 3$

删除操作摊销代价分析:

情况 1: 删除后 $\alpha > 1/4$ (不收缩)

- 实际代价: $c_i = 1$
- 势能变化: $\Delta\Phi = -2$ 或 $+1$ (取决于 α 范围)
- 摊销代价: $\hat{c}_i = O(1)$

情况 2: 删除后 $\alpha = 1/4$ (触发收缩)

- 类似分析, 摊销代价仍为 $O(1)$

结论: 在新方案下, 插入和删除的摊销时间复杂度均为 $O(1)$, 且避免了抖动问题。

第 7 题

题目: 类似蒙特卡洛随机算法中的素数测试问题。

Miller-Rabin 素数测试算法

问题: 给定一个大整数 n , 判断 n 是否为素数。

算法类型: 蒙特卡洛算法

- 如果返回“合数”, 则 n 一定是合数 (确定性)
- 如果返回“可能是素数”, 则 n 可能是素数, 也可能是合数 (有错误概率)

算法原理

基于费马小定理和二次探测:

费马小定理: 如果 n 是素数, a 是小于 n 的正整数, 则:

$$a^{n-1} \equiv 1 \pmod{n}$$

Miller-Rabin 测试:

将 $n - 1$ 写成 $2^s \cdot d$ 的形式, 其中 d 是奇数。

对于随机选择的 $a \in [2, n - 2]$, 如果 n 是素数, 则必满足以下之一:

1. $a^d \equiv 1 \pmod{n}$
2. 存在 $r \in [0, s-1]$, 使得 $a^{2^r \cdot d} \equiv -1 \pmod{n}$

如果都不满足, 则 n 一定是合数。

算法伪代码

Algorithm Miller-Rabin 素数测试

Input: 待测试整数 $n > 2$, 测试轮数 k

Output: "素数" 或 "合数"

```

1: if  $n = 2$  or  $n = 3$  then
2:   return "素数"
3: if  $n < 2$  or  $n \bmod 2 = 0$  then
4:   return "合数"
5: 将  $n - 1$  写成  $2^s \cdot d$  形式 ( $d$  为奇数)
6: for  $i = 1$  to  $k$  do                                     ▷ 执行  $k$  轮测试
7:    $a \leftarrow$  在  $[2, n - 2]$  中随机选择
8:    $x \leftarrow a^d \bmod n$                                    ▷ 快速幂计算
9:   if  $x = 1$  or  $x = n - 1$  then
10:    continue                                             ▷ 通过本轮测试
11:   for  $r = 1$  to  $s - 1$  do
12:      $x \leftarrow x^2 \bmod n$ 
13:     if  $x = n - 1$  then
14:       break                                             ▷ 通过本轮测试
15:   if  $x \neq n - 1$  then
16:     return "合数"                                       ▷ 未通过测试
17: return "可能是素数"

```

复杂度分析

时间复杂度:

- 单轮测试: $O(\log n)$ (快速幂的复杂度)
- k 轮测试: $O(k \log n)$

错误概率分析

定理: 如果 n 是合数, 则至少 $3/4$ 的 $a \in [2, n - 2]$ 会使测试失败 (检测出合数)。

因此，单轮测试的错误概率（将合数误判为素数） $\leq 1/4$ 。

k 轮独立测试的错误概率：

$$P(\text{错误}) \leq \left(\frac{1}{4}\right)^k = \frac{1}{4^k}$$

实践中的选择：

- $k = 10$ ：错误概率 $\leq 10^{-6}$
- $k = 20$ ：错误概率 $\leq 10^{-12}$

结论：Miller-Rabin 是一个高效的概率算法，通过增加测试轮数可以将错误概率降到任意小。

第 8 题

题目：回忆近似算法中的最短并行调度问题，现在对该方法改进，将各带权区间按照权值从大到小排列，顺序将各区间分配给当时各机器总时间最短的机器上（与讲过的处理方法区别仅在于：以前是将各区间随机排列的），证明这种近似算法的近似比为 $3/2$ 。

问题描述

最短并行调度问题 (Makespan Scheduling)：

- 输入： n 个任务，处理时间分别为 $t_1, t_2, \dots, t_n$ ； m 台机器
- 目标：将任务分配到机器上，最小化完成时间 (makespan)
- $\text{makespan} = \max_{i=1}^m T_i$ ，其中 T_i 是机器 i 的总处理时间

改进的贪心算法

算法描述：

1. 将任务按处理时间从大到小排序： $t_1 \geq t_2 \geq \dots \geq t_n$
2. 依次考虑每个任务，将其分配给当前负载最小的机器

这被称为 LPT (Longest Processing Time) 算法。

算法伪代码

Algorithm LPT 调度算法

Input: 任务处理时间 $t[1..n]$, 机器数 m

Output: 每台机器的任务分配

- 1: 将任务按 t_i 从大到小排序
 - 2: 初始化 $Load[1..m] = 0$ ▷ 每台机器的当前负载
 - 3: **for** $i = 1$ to n **do**
 - 4: $j \leftarrow \arg \min_{k=1}^m Load[k]$ ▷ 选择负载最小的机器
 - 5: 将任务 i 分配给机器 j
 - 6: $Load[j] \leftarrow Load[j] + t[i]$
 - 7: **return** makespan = $\max_{k=1}^m Load[k]$
-

近似比证明：近似比为 $3/2$

记号：

- OPT : 最优解的 makespan
- ALG : LPT 算法的 makespan
- T_{max} : 机器的最大负载 (ALG 的值)
- t_{last} : 最后一个被分配到负载最大机器上的任务

下界 1:

$$OPT \geq \frac{\sum_{i=1}^n t_i}{m}$$

(最优解至少要平均分配)

下界 2:

$$OPT \geq t_1 \geq t_{last}$$

(最优解至少要能处理最大的任务)

关键观察:

假设机器 j 达到了最大负载 T_{max} , 且 t_{last} 是最后一个分配到机器 j 的任务。

在分配 t_{last} 之前, 机器 j 的负载最小, 因此:

$$T_{max} - t_{last} \leq \frac{\sum_{i: t_i > t_{last}} t_i}{m}$$

情况分析:

情况 1: 如果 $t_{last} \leq OPT/3$

则：

$$\begin{aligned}
 ALG &= T_{max} \\
 &= (T_{max} - t_{last}) + t_{last} \\
 &\leq \frac{\sum_{i=1}^n t_i}{m} + t_{last} \\
 &\leq OPT + \frac{OPT}{3} \\
 &= \frac{4OPT}{3}
 \end{aligned}$$

近似比 $\leq 4/3 < 3/2$ 。

情况 2： 如果 $t_{last} > OPT/3$

由于任务按降序排列，所有 $t_i \geq t_{last} > OPT/3$ 。

这意味着每个这样的任务都不能与其他任务放在同一台机器上（否则负载 $> 2 \cdot OPT/3$ 可能超过 OPT ）。

因此，这样的任务数量 $\leq m$ （每台机器最多一个）。

如果 t_{last} 是第 k 个任务，则 $k \leq m$ 。

对于 $k \leq m$ ，LPT 的分配结果是：前 m 个任务分别放在 m 台机器上，后续任务依次添加。

在这种情况下：

$$ALG = \max(t_1, t_2 + t_{m+1}, \dots) \leq t_1 + t_{m+1}$$

但由于 $t_1 \geq t_2 \geq \dots \geq t_m > OPT/3$ 且 $k \leq m$ ：

如果所有前 m 个任务都 $> OPT/3$ ，则：

$$m \cdot \frac{OPT}{3} < \sum_{i=1}^m t_i \leq \sum_{i=1}^n t_i \leq m \cdot OPT$$

这是可能的。但关键是，如果 $t_{last} > OPT/3$ ，则：

$$ALG \leq t_1 + t_{m+1} \leq OPT + \frac{OPT}{3} = \frac{4OPT}{3}$$

（因为 $t_{m+1} \leq OPT/3$ ）

更严格的分析表明，LPT 的近似比为：

$$\frac{ALG}{OPT} \leq \frac{3}{2} - \frac{1}{2m}$$

结论： LPT 算法的近似比为 $3/2$ （当 $m \rightarrow \infty$ 时）。

第 9 题

题目： 现有最大路径问题，在无向图（非完全图）上找两点之间的最大路径，请判断该问题是 NP 问题还是 NPC 问题，如果是 NPC 问题，请用模拟退火算法框架给出该问题的解决方法。

问题判定：NPC 问题

最长简单路径问题 (Longest Simple Path): 在无向图中找两点之间的最长简单路径 (不重复访问顶点)。

判定:

1. 该问题属于 NP:

- 给定一条路径 (证书), 可以在多项式时间内验证:
- 检查路径连接给定的两点
- 检查路径上的顶点不重复
- 计算路径长度
- 验证复杂度: $O(n)$

2. 该问题是 NP-Hard:

- **哈密尔顿路径问题**可以归约到最长路径问题
- 哈密尔顿路径: 判断是否存在经过所有顶点恰好一次的路径
- 归约: 在图中是否存在长度为 $n - 1$ 的路径 (n 个顶点)
- 因此, 最长路径问题至少和哈密尔顿路径问题一样难
- 而哈密尔顿路径问题是 NPC 问题

结论: 最长路径问题是 NPC 问题。

模拟退火算法求解

算法设计

1. 解的表示:

- 一个解表示为从起点到终点的一条路径
- 编码: 顶点序列 $[v_1, v_2, \dots, v_k]$, 其中 v_1 是起点, v_k 是终点

2. 目标函数:

$$f(\text{path}) = \text{路径长度 (经过的边数或边权之和)}$$

目标: 最大化 f 。

3. 初始解:

- 使用 BFS 或 DFS 找一条从起点到终点的简单路径
- 或随机生成一条路径

4. 邻域操作 (生成新解):

从当前路径生成邻域解的方法:

1. **插入顶点:** 在路径中间插入一个不在路径上的顶点 (如果可行)
2. **删除顶点:** 从路径中删除一个中间顶点 (保持连通性)
3. **替换顶点:** 用不在路径上的顶点替换路径中的某个顶点
4. **2-opt 交换:** 选择路径上的两段, 尝试重新连接

5. 接受准则:

使用 Metropolis 准则:

- 如果新解更优 ($f_{new} < f_{current}$), 接受新解
- 如果新解更差, 以概率 $e^{(f_{new}-f_{current})/T}$ 接受 (T 为温度)

6. 温度调度:

- 初始温度 T_0 : 较高 (如 100)
- 降温策略: $T_{k+1} = \alpha \cdot T_k$, 其中 $\alpha \in (0.9, 0.99)$
- 终止温度 T_{min} : 很小 (如 0.01)

算法伪代码

Algorithm 模拟退火求解最长路径

Input: 无向图 $G = (V, E)$, 起点 s , 终点 t

Output: 近似最长路径

```

1:  $current \leftarrow$  初始路径 (从  $s$  到  $t$ )
2:  $best \leftarrow current$ 
3:  $T \leftarrow T_0$  ▷ 初始温度
4: while  $T > T_{min}$  do
5:   for  $iter = 1$  to  $L$  do ▷ 每个温度下的迭代次数
6:      $neighbor \leftarrow$  从  $current$  生成邻域解
7:     if  $neighbor$  是有效路径 then
8:        $\Delta f \leftarrow f(neighbor) - f(current)$ 
9:       if  $\Delta f > 0$  then ▷ 新解更优
10:         $current \leftarrow neighbor$ 
11:        if  $f(current) > f(best)$  then
12:           $best \leftarrow current$ 
13:        else ▷ 新解更差
14:           $p \leftarrow e^{\Delta f / T}$ 
15:          if  $random() < p$  then ▷ 以概率接受
16:             $current \leftarrow neighbor$ 
17:    $T \leftarrow \alpha \cdot T$  ▷ 降温
18: return  $best$ 

```

参数设置建议

- 初始温度 T_0 : 100-1000 (取决于目标函数的范围)
- 降温系数 α : 0.90-0.99
- 每温度迭代次数 L : $10n$ 到 $100n$ (n 为顶点数)
- 终止温度 T_{min} : 0.01-0.1

算法特点

- **优点:** 能跳出局部最优, 探索更广的解空间
- **缺点:** 不保证找到全局最优解, 参数敏感
- **适用性:** 对于 NPC 问题, 是一种实用的启发式方法